

5.5

区间 DP



扫一扫



视频讲解

区间 DP 是常见的 DP 应用场景。区间 DP 也是线性 DP，它把区间当成 DP 的阶段，用区间的两个端点描述状态和处理状态的转移。区间 DP 的主要思想是先在小区间进行 DP 得到最优解，然后再合并小区间的最优解求得大区间的最优解。解题时，先解决小区间问题，然后合并小区间，得到更大的区间，直到解决最后的大区间问题。合并的操作一般是把左右两个相邻的子区间合并。

区间 DP 的计算量比较大。一个长度为 n 的区间，编程时，区间 DP 至少需要两层 for 循环，第 1 层的 i 从区间的首部或尾部开始；第 2 层的 j 从 i 开始到结束， i 和 j 一起枚举出所有的子区间。所以，区间 DP 的复杂度大于 $O(n^2)$ ，一般为 $O(n^3)$ 。



很多区间 DP 问题可以用四边形不等式优化,把复杂度降为 $O(n^2)$ 。

5.5.1 石子合并问题和两种模板代码

区间 DP 的经典例子是石子合并问题,下面用这个例子解释区间 DP 的概念,并给出模板代码。



例 5.11 石子合并

问题描述:有 n 堆石子排成一排,每堆石子有一定的数量。将 n 堆石子合并成一堆,每次只能合并相邻的两堆石子,合并的花费为这两堆石子的总数。经过 $n-1$ 次合并后成为一堆,求最小总花费。

输入:第 1 行输入整数 n ,表示有 n 堆石子。第 2 行输入 n 个数,分别表示这 n 堆石子的数目。

输出:最小总花费。

输入样例:

3
2 4 5

输出样例:

17



样例的计算过程是第 1 次合并 $2+4=6$;第 2 次合并 $6+5=11$;总花费为 $6+11=17$ 。

本题不能用贪心法求解,下面指出原因。先回顾贪心法,它的应用场景是从局部最优扩展到全局最优。

(1) 如果石子堆没有顺序,可以任意合并,用贪心法每次选择最小的两堆合并。

(2) 本题要求只能合并相邻的两堆,不能用贪心法。贪心操作是每次合并时找石子数相加最少的两堆相邻石子。例如,环形石子堆,初始是 $\{2,4,7,5,4,3\}$,用贪心法得到最小花费 64,但是另一种方法的结果为 63,如表 5.2 所示。

表 5.2 贪心法对比

步骤	贪心法(方法 1)		方法 2	
	花费	剩余石子堆	花费	剩余石子堆
初始	—	2,4,7,5,4,3	—	2,4,7,5,4,3
1	5	5,4,7,5,4	6	6,7,5,4,3
2	9	9,7,5,4	13	13,5,4,3
3	9	9,7,9	7	13,5,7
4	16	16,9	12	13,12
5	25	25	25	25
总花费	64		63	

下面用 DP 求解。

定义 $dp[i][j]$ 为合并第 i 堆到第 j 堆的最小花费。

状态转移方程为

$$dp[i][j] = \min\{dp[i][k] + dp[k+1][j] + w[i][j]\}, \quad i \leq k < j$$

$dp[1][n]$ 就是答案。方程中的 $w[i][j]$ 表示从第 i 堆到第 j 堆的石子总数。

按自顶向下的思路分析状态转移方程,很容易理解。计算大区间 $[i, j]$ 的最优值时,合并它的两个子区间 $[i, k]$ 和 $[k+1, j]$, 对所有可能的合并 ($i \leq k < j$, 即 k 在 i 和 j 之间滑动), 采用最优的合并。子区间再分解为更小的区间,最小的区间 $[i, i+1]$ 只包含两堆石子。

编程用自底向上递推的方法,先在小区间进行 DP 得到最优解,然后再逐步合并小区间为大区间。下面给出求 $dp[i][j]$ 的代码,其中包含 i, j, k 的 3 层循环,时间复杂度为 $O(n^3)$ 。第 1 个循环 j 是区间终点,第 2 个循环 i 是区间起点,第 3 个循环 k 在区间内滑动。注意,起点 i 应该从 $j-1$ 开始递减,也就是从最小的区间 $[j-1, j]$ 开始,逐步扩大区间。 i 不能从 1 开始递增,那样就是从大区间到小区间了。

```

1  for(int i = 1; i <= n; i++)  dp[i][i] = 0; //n 为石子堆数,初始化
2  for(int j = 2; j <= n; j++)      // 区间[i, j]的终点 j, i < j <= n
3      for(int i = j-1; i >= 1; i--) { // 区间[i, j]的起点 i
4          dp[i][j] = INF;           // 初始化为极大值
5          for(int k = i; k < j; k++) // 大区间[i, j]从小区间[i, k]和[k+1, j]转移而来
6              dp[i][j] = min(dp[i][j], dp[i][k] + dp[k+1][j] + w[i][j]);
7      }

```

下面给出另一种写法,它的 i, j, k 都是递增的,更容易理解,推荐使用这种写法。代码中用了个辅助变量 len ,它等于当前处理的区间 $[i, j]$ 的长度。 $dp[i][j]$ 是大区间,它需要从小区间 $dp[i][k]$ 和 $dp[k+1][j]$ 转移而来,所以应该先计算出小区间,才能根据小区间计算出大区间。 len 就起到了这个作用,从最小的区间 $len=2$ 开始,此时区间 $[i, j]$ 为 $[i, i+1]$;最后是最大区间 $len=n$,此时区间 $[i, j]$ 为 $[1, n]$ 。

```

1  for(int i = 1; i <= n; i++)  dp[i][i] = 0;
2  for(int len = 2; len <= n; len++) // len 为区间[i, j]的长度,从小区间扩展到大区间
3      for(int i = 1; i <= n-len+1; i++) { // 区间起点 i
4          int j = i + len - 1;           // 区间终点 j, i < j <= n
5          dp[i][j] = INF;
6          for(int k = i; k < j; k++)
7              dp[i][j] = min(dp[i][j], dp[i][k] + dp[k+1][j] + w[i][j]);
8      }

```

上述代码的复杂度为 $O(n^3)$,不太好。区间 DP 常常可以用四边形不等式进行优化,把复杂度性能提升到 $O(n^2)$,详见 5.10 节。经典例题“石子合并”也在 5.10 节进行了更多讲解。当然,不是所有的区间 DP 都能做四边形不等式优化。

5.5.2 区间 DP 例题

下面给出几道经典区间 DP 例题。请读者在看题解之前,自己思考并写出代码,一定会大有收获。

1. hdu 2476



例 5.12 String painter(hdu 2476)

问题描述：给定两个长度相等的字符串 A 、 B ，由小写字母组成。一次操作，允许把 A 中的一个连续子串(区间)都转换为某个字符(就像用刷子刷成一样的字符)。要把 A 转换为 B ，问最少操作数是多少？

输入：第 1 行输入字符串 A ，第 2 行输入字符串 B 。两个字符串的长度不大于 100。

输出：一个表示答案的整数。

输入样例：

zzzzzfzzzzz
abcdefedcba

输出样例：

6



提示 第 1 次把 zzzzzfzzzzz 转换为 aaaaaaaaaa，第 2 次转换为 abbbbbbbba，第 3 次转换为 abccccccba...

这道经典例题能帮助读者深入理解区间 DP 是如何构造和编码的。

1) 从空白串转换到 B

先考虑一个简单的问题：从空白串转换到 B 。为方便阅读代码，把字符串存储为 $B[1] \sim B[n]$ ，不用 $B[0] \sim B[n-1]$ ，编码时这样输入：scanf("%s%s", $A+1$, $B+1$)。

如何定义 DP 状态？可以定义 $dp[i]$ 表示在区间 $[1, i]$ 内转换为 B 的最少操作次数。或者更进一步，定义 $dp[i][j]$ 表示在区间 $[i, j]$ 内从空白串转换到 B 时的最少操作次数。重点是区间 $[i, j]$ 两端的字符 $B[i]$ 和 $B[j]$ ，分析以下两种情况。

(1) $B[i] = B[j]$ 。第 1 次用 $B[i]$ 把区间 $[i, j]$ 刷一遍，这个刷法肯定是最优的。如果分别去掉两个端点，得到两个区间 $[i+1, j]$ 和 $[i, j-1]$ ，这两个区间的最少操作次数相等，也等于原区间 $[i, j]$ 的最少操作次数。例如， $B = "abbba"$ ，先用 "a" 全部刷一遍，再刷一次 "bbb"，共刷两次。如果去掉第 1 个 "a"，剩下的 "bbba" 也是刷两次。

(2) $B[i] \neq B[j]$ 。因为两个端点不同，至少要各刷一次。用标准的区间操作，把区间分成 $[i, k]$ 和 $[k+1, j]$ 两部分，枚举最少操作次数。

2) 从 A 转换到 B

如何求 $dp[1][j]$ ？观察 A 和 B 相同位置的字符，分析以下两种情况。

(1) $A[j] = B[j]$ 。这个字符不用转换，有 $dp[1][j] = dp[1][j-1]$ 。

(2) $A[j] \neq B[j]$ 。仍然用标准的区间 DP，把区间分成 $[1, k]$ 和 $[k+1, j]$ 两部分，枚举最少操作次数。这里利用了从空白串转换到 B 的结果，当区间 $[k+1, j]$ 内 A 和 B 的字符不同时，从 A 转换到 B 与从空白串转换到 B 是等价的。

下面给出 hdu 2476 的代码，其中，从空白串转换到 B 的代码完全套用了石子合并问题的编码。


```

1  #include <bits/stdc++.h>
2  using namespace std;
3  char A[105], B[105];
4  int dp[105][105];
5  const int INF = 0x3f3f3f3f;
6  int main() {
7      while(~scanf("%s%s", A+1, B+1)){
8          int n = strlen(A+1);          //输入 A, B
9          for(int i = 1; i <= n; i++)    dp[i][i] = 1; //初始化
10         //先从空白串转换到 B
11         for(int len = 2; len <= n; len++){
12             for(int i = 1; i <= n - len + 1; i++){
13                 int j = i + len - 1;
14                 dp[i][j] = INF;
15                 if(B[i] == B[j])        //区间[i, j]两端的字符相同, B[i] = B[j]
16                     dp[i][j] = dp[i+1][j]; //或者 dp[i][j] = dp[i][j-1]
17                 else                    //区间[i, j]两端的字符不同 B[i] ≠ B[j]
18                     for(int k = i; k < j; k++){
19                         dp[i][j] = min(dp[i][j], dp[i][k] + dp[k+1][j]);
20                     }
21         //下面从 A 转换到 B
22         for(int j = 1; j <= n; ++j){
23             if(A[j] == B[j]) dp[1][j] = dp[1][j-1]; //字符相同, 不用转换
24             else
25                 for(int k = 1; k < j; ++k)
26                     dp[1][j] = min(dp[1][j], dp[1][k] + dp[k+1][j]);
27         }
28         printf("%d\n", dp[1][n]);
29     }
30     return 0;
31 }

```

2. hdu 4283



例 5.13 You are the one(hdu 4283)

问题描述: n 个男孩去相亲, 排成一队上场。大家都不想等, 排队越靠后越愤怒。每个人的耐心不同, 用 D 表示火气, 设男孩 i 的火气为 D_i , 他排在第 k 个时, 愤怒值为 $(k-1)D_i$ 。

主持人不想看到会场气氛紧张。他安排了一间黑屋, 可以调整这排男孩上场的顺序, 屋子很狭长, 先进去的男孩最后出来(相当于一个堆栈)。例如, 当男孩 A 排到时, 如果他后面的男孩 B 火气更大, 就把 A 送进黑屋, 让 B 先上场。一般情况下, 那些火气小的男孩要多等等, 让火气大的占便宜。不过, 零脾气的你也不一定吃亏, 如果你原本排在倒数第 2 个, 而最后一个男孩脾气最坏, 主持人为了让这个坏家伙第 1 个上场, 把其他人全赶进了黑屋, 结果你就排在了黑屋的第 1 名, 将第 2 个上场。

注意, 每个男孩都要进出黑屋。

对所有男孩的愤怒值求和, 求所有可能情况的最小总愤怒值。

输入：第 1 行输入一个整数 T ，即测试用例的数量。对于每种情况，第 1 行输入 n ($0 < n \leq 100$)，后面 n 行中，输入整数 $D_1 \sim D_n$ 表示男孩的火气 ($0 \leq D_i \leq 100$)。

输出：对每个用例，输出最小总愤怒值之和。

读者可以试试用栈来模拟，这几乎是不可能的。这是一道区间 DP 例题，巧妙地利用了栈的特性。

定义 $dp[i][j]$ 表示从第 i 个人到第 j 个人，即区间 $[i, j]$ 的最小总愤怒值。

由于栈的存在，本题的区间 $[i, j]$ 的分割点 k 比较特殊。分割时，总是用区间 $[i, j]$ 的第 1 个元素 i 把区间分成两部分，让 i 第 k 个从黑屋出来上场，即第 k 个出栈。根据栈的特性，若第 1 个元素 i 第 k 个出栈，则第 $2 \sim k-1$ 个元素肯定在第 1 个元素之前出栈，第 $k+1 \sim j$ 个元素肯定在第 k 个之后出栈^①。

例如，5 个人的排队序号是 1, 2, 3, 4, 5。如果第 1 ($i=1$) 个人要第 3 ($k=3$) 个出场，那么栈的操作是这样：1 号进栈 $\rightarrow$ 2 号进栈 $\rightarrow$ 3 号进栈 $\rightarrow$ 3 号出栈 $\rightarrow$ 2 号出栈 $\rightarrow$ 1 号出栈。2 号和 3 号在 1 号之前出栈，1 号第 3 个出栈，4 号和 5 号在 1 号后面出栈。

分割点 k ($1 \leq k \leq j-i+1$) 把区间划分成两段，即 $dp[i+1][i+k-1]$ 和 $dp[i+k][j]$ 。 $dp[i][j]$ 的计算分为以下 3 部分。

(1) $dp[i+1][i+k-1]$ 。原来 i 后面的 $k-1$ 个人现在排到 i 前面了。

(2) $D[i](k-1)$ 。第 i 个人往后挪了 $k-1$ 个位置，愤怒值增加 $D[i](k-1)$ 。

(3) $dp[i+k][j] + k(\text{sum}[j] - \text{sum}[i+k-1])$ 。第 k 个位置后面的人，即区间 $[i+k, j]$ 的人，由于都在前 k 个人之后，相当于从区间的第 1 个位置往后挪了 k 个位置，所以整体愤怒值要加上 $k(\text{sum}[j] - \text{sum}[i+k-1])$ 。其中， $\text{sum}[j] = \sum_{i=1}^j D_i$ 为 $1 \sim j$ 所有人火气值的和， $\text{sum}[j] - \text{sum}[i+k-1]$ 是区间 $[i+k, j]$ 内人的火气值的和。

代码完全套用石子合并的模板。其中，DP 方程给出了两种写法，对照看更清晰。

```

1  for(int len=2; len<=n; len++)
2      for(i=1; i<=n-len+1; i++) {
3          j = len + i - 1;
4          dp[i][j] = INF;
5          for(int k=1; k<=j-i+1; k++) //k: i 往后挪了 k 位, 这样写容易理解
6              dp[i][j] = min(dp[i][j], dp[i+1][i+k-1] + D[i] * (k-1)
7                             + dp[i+k][j] + k * (sum[j] - sum[i+k-1])); //DP 方程
8          //for(int k=i; k<=j; k++) //或者这样写, k 为整个队伍的绝对位置
9          //    dp[i][j] = min(dp[i][j], dp[i+1][k] + D[i] * (k-i)
10             //                + dp[k+1][j] + (k-i+1) * (sum[j] - sum[k]));
11      }

```

5.5.3 二维区间 DP

前面例子中的区间 $[i, j]$ 可以看作在一条直线上移动，即一维 DP。下面给出一道二维区间 DP 的例题，它的区间同时在两个方向移动。

^① https://blog.csdn.net/weixin_41707869/article/details/99686868



例 5.14 Rectangle painting 1(CF1199F)^①

问题描述：有一个 $n \times n$ 大小的方格图，某些方格初始为黑色，其余为白色。一次操作，可以选定一个 $h \times w$ 的矩形，把其中所有方格涂成白色，代价是 $\max(h, w)$ 。要求用最小的代价把所有方格涂成白色。

输入：第 1 行输入整数 n ，表示方格的大小。后面有 n 行，每行输入长度为 n 的串，包含符号 '.' 和 '#'，'.' 表示白色，'#' 表示黑色。第 x 行第 y 列的坐标是 (x, y) 。 $n \leq 50$ 。

输出：打印一个整数，表示把所有方格涂成白色的最小代价。

输入样例：

```
5
#...#
#.#.
.....
.#...
#....
```

输出样例：

```
5
```

设矩形区域从左下角坐标 (x_1, y_1) 到右上角坐标 (x_2, y_2) 。定义状态 $dp[x_1][y_1][x_2][y_2]$ 表示把这个区域涂成白色的最小代价。

这个区域可以分别按 x 轴或按 y 轴分割成两个矩形，遍历所有可能的分割，求最小代价。从 x 方向看，是一个区间 DP；从 y 方向看，也是一个区间 DP。

代码可以完全套用一维 DP 的模板，分别在两个方向上操作。

(1) x 方向，把区间分为 $[x_1, k]$ 和 $[k+1, x_2]$ 。状态转移方程为

$$dp[x_1][y_1][x_2][y_2] = \min(dp[x_1][y_1][x_2][y_2], dp[x_1][y_1][k][y_2] + dp[k+1][y_1][x_2][y_2])$$

(2) y 方向，把区间分为 $[y_1, k]$ 和 $[k+1, y_2]$ 。状态转移方程为

$$dp[x_1][y_1][x_2][y_2] = \min(dp[x_1][y_1][x_2][y_2], dp[x_1][y_1][x_2][k] + dp[x_1][k+1][x_2][y_2])$$

下面给出代码，有 5 层循环，时间复杂度为 $O(n^5)$ 。对比一维区间 DP，有 3 层循环，复杂度为 $O(n^3)$ 。

```
1 //代码改写自 https://www.cnblogs.com/zsben991126/p/11643832.html
2 #include <bits/stdc++.h>
3 using namespace std;
4 #define N 55
5 int dp[N][N][N][N];
6 char mp[N][N]; //方格图
7 int main(){
8     int n; cin >> n;
9     for(int i = 1; i <= n; i++)
10         for(int j = 1; j <= n; j++) cin >> mp[i][j];
```

^① <https://codeforces.com/contest/1199/problem/F>，如果 CodeForces 很慢，用代理提交 (<https://vjudge.net/problem/CodeForces-1199F>)。


```

11     for(int i = 1; i <= n; i++)
12         for(int j = 1; j <= n; j++)
13             if(mp[i][j] == '.') dp[i][j][i][j] = 0;        //白格不用涂
14             else dp[i][j][i][j] = 1;                        //黑格(i, j)涂成白色,需一次
15     for(int lenx = 1; lenx <= n; lenx++)                    //len 从 1 开始,不是 2,因为有 x 和 y 两个方向
16         for(int leny = 1; leny <= n; leny++)
17             for(int x1 = 1; x1 <= n - lenx + 1; x1++)
18                 for(int y1 = 1; y1 <= n - leny + 1; y1++){
19                     int x2 = x1 + lenx - 1;                //x1 为 x 轴起点, x2 为 x 轴终点
20                     int y2 = y1 + leny - 1;                //y1 为 y 轴起点, y2 为 y 轴终点
21                     if(x1 == x2 && y1 == y2) continue;    //lenx = 1 且 leny = 1 的情况
22                     dp[x1][y1][x2][y2] = max(abs(x1 - x2), abs(y1 - y2)) + 1; //初始值
23                     for(int k = x1; k < x2; k++) //枚举 x 方向, y 不变, 区间[x1, k] + [k + 1, x2]
24                         dp[x1][y1][x2][y2] = min(dp[x1][y1][x2][y2],
25                                                     dp[x1][y1][k][y2] + dp[k + 1][y1][x2][y2]);
26                     for(int k = y1; k < y2; k++) //枚举 y 方向, x 不变, 区间[y1, k] + [k + 1, y2]
27                         dp[x1][y1][x2][y2] = min(dp[x1][y1][x2][y2],
28                                                     dp[x1][y1][x2][k] + dp[x1][k + 1][x2][y2]);
29                 }
30     cout << dp[1][1][n][n];
31 }

```

【习题】

- (1) 力扣: <https://leetcode-cn.com/circle/article/NfHhXD/>。
- (2) 区间 DP(kuangbin 带你飞): <https://vjudge.net/contest/77874>。
- (3) 洛谷 P1880/P3146/P1063/P1005/P4170/P4302/P2466。

扫一扫



视频讲解

5.6

树形 DP



在树这种数据结构上做 DP 很常见: 给出一棵树, 要求以最少代价(或最大收益)完成给定的操作。在树上做 DP 显得非常自然, 因为树本身有“子结构”性质(树和子树), 具有递归性, 符合 5.1.2 节提到的“记忆化递归”的思路, 所以树形 DP 一般就这样编程。

基于树的解题步骤一般是先把树转换为有根树(如果是几棵互不连通的树, 就加一个虚拟根, 它连接所有孤立的树), 然后在树上做 DFS, 递归到最底层的叶子节点, 再一层层返回信息更新至根节点。显然, 树上 DP 所操作的就是这一层层返回的信息。不同的题目需要灵活设计不同的 DP 状态和转移方程。

树需要用数据结构来存储。关于树的存储方式, 请提前阅读本书 10.1 节“图的存储”。

5.6.1 树形 DP 的基本操作

先看一道简单的入门题, 了解树的存储以及如何在树上设计 DP 和进行状态转移。请读者特别注意 DP 设计时的两种处理方法: 二叉树、多叉树。